

Internals: the code, line by line

Estrogen and aneurysmal SAH: companion code

N. Pacheco

Contents

Internals: the code, line by line	1
1. The MR estimators (<code>mr/src/estrogen_mr/methods.py</code>)	1
2. Instruments: clumping and harmonization (<code>mr/src/estrogen_mr/instruments.py</code>) . .	4
3. Multivariable MR (<code>mr/src/estrogen_mr/mvmr.py</code>)	5
4. Quality control, power, equivalence (<code>mr/src/estrogen_mr/qc.py</code>)	6
5. The ICU identification argument (<code>icu/src/estrogen_asah_dci/analysis/multiverse.py</code>)	6

Internals: the code, line by line

A teaching walkthrough of the analysis code. It assumes you can read Python but not that you know Mendelian randomization or the statistics. We go through the core files line by line, deriving the maths where it matters. File paths are relative to the repository root.

Contents: 1. The MR estimators, `mr/src/estrogen_mr/methods.py` 2. Instruments: clumping and allele harmonization, `mr/src/estrogen_mr/instruments.py` 3. Multivariable MR, `mr/src/estrogen_mr/mvmr.py` 4. Quality control, power, equivalence, `mr/src/estrogen_mr/qc.py` 5. The ICU identification argument, `icu/src/estrogen_asah_dci/analysis/multiverse.py`

1. The MR estimators (`mr/src/estrogen_mr/methods.py`)

Every estimator takes four arrays with one entry per genetic variant (SNP): b_x (the SNP's effect on the exposure), s_x (its standard error), b_y (the SNP's effect on the outcome), s_y (its standard error). The idea of MR: if a SNP raises the exposure by b_x , and the exposure causally raises the outcome by some amount θ , then the SNP should raise the outcome by $\theta * b_x$. So a plot of b_y against b_x should be a straight line through the origin with slope θ . Each estimator is a different way to fit that slope.

Inverse-variance weighted (IVW)

```
def ivw(bx, sx, by, sy) -> MRResult:
    bx, sx, by, sy = map(np.asarray, (bx, sx, by, sy))
    w = 1.0 / sy**2
```

```
est = np.sum(w * bx * by) / np.sum(w * bx**2)
se = np.sqrt(1.0 / np.sum(w * bx**2))
return _ci(est, se, "IVW", len(bx))
```

- $w = 1.0 / sy^2$, each SNP's weight is one over its outcome variance. A SNP whose outcome effect is measured precisely (small sy) counts more. This is the “inverse-variance” in the name.
- $est = \text{sum}(w \cdot bx \cdot by) / \text{sum}(w \cdot bx^2)$, this is the weighted least-squares slope of a line through the origin. If you minimise $\text{sum}(w \cdot (by - \theta \cdot bx)^2)$ over θ , set the derivative to zero, you get exactly $\theta = \text{sum}(w \cdot bx \cdot by) / \text{sum}(w \cdot bx^2)$. So IVW is just a weighted regression of outcome effects on exposure effects, forced through zero.
- $se = \sqrt{1 / \text{sum}(w \cdot bx^2)}$, the standard error of that slope. It shrinks when you have many strong instruments (large bx) measured precisely (small sy).
- `_ci(...)` wraps the estimate into a result object with a 95% interval and a p-value; see below.

The estimate is on the log-odds scale because the outcome GWAS is a case/control study, so $\exp(est)$ is the causal odds ratio per unit of exposure. That exponentiation happens in `MRResult.ods_ratio`.

The result wrapper

```
def _ci(est, se, method, n, extra_dof=0) -> MRResult:
    z = est / se if se > 0 else np.nan
    p = float(2 * stats.norm.sf(abs(z))) if se > 0 else np.nan
    return MRResult(method, float(est), float(se),
        float(est - 1.96 * se), float(est + 1.96 * se), p, n)
```

- $z = est / se$, a standard z-score: how many standard errors the estimate sits from zero.
- `stats.norm.sf(abs(z))` is the upper tail of the normal (`sf` = survival function = $1 - \text{cdf}$); doubling it gives a two-sided p-value.
- $est - 1.96 \cdot se$ and $est + 1.96 \cdot se$ are the 95% confidence limits on the log-odds scale; the `MRResult` exponentiates them for the odds-ratio interval.

MR-Egger

IVW assumes every SNP affects the outcome only through the exposure. MR-Egger relaxes that by letting the line have an intercept: if the SNPs have a shared “pleiotropic” effect that bypasses the exposure, the intercept picks it up, and the slope stays an unbiased causal estimate.

```
sign = np.sign(bx)
bx_o, by_o = bx * sign, by * sign
```

- MR-Egger requires the exposure effects to all point the same way, so we flip the sign of any SNP with a negative bx (and flip its by to match). This is cosmetic re-coding of the effect allele; it does not change the biology.

```
X = np.column_stack([np.ones_like(bx_o), bx_o])
W = np.diag(w)
```

```
xtwx = X.T @ W @ X
beta = np.linalg.solve(xtwx, X.T @ W @ by_o)
```

- X is a design matrix with two columns: a column of ones (for the intercept) and bx_o (for the slope). This is ordinary weighted linear regression, $by \sim 1 + bx$.
- $\text{beta} = \text{solve}(X'WX, X'W y)$ is the closed-form weighted least-squares solution. $\text{beta}[0]$ is the intercept, $\text{beta}[1]$ is the slope (the causal estimate).

```
resid = by_o - X @ beta
dof = max(len(bx_o) - 2, 1)
sigma2 = (resid @ (w * resid)) / dof
cov = sigma2 * np.linalg.inv(xtwx)
```

- resid are the leftover distances from the fitted line.
- sigma2 is the weighted residual variance, dividing by $n - 2$ degrees of freedom (two parameters were fitted). Scaling the covariance by this term is what makes MR-Egger use random-effects standard errors, which is standard practice.
- cov is the covariance matrix of (intercept, slope).

```
p_int = float(2 * stats.t.sf(abs(intercept / se_int), dof)) ...
```

- The intercept's p-value uses a t-distribution (small number of SNPs). A small intercept_p is a warning of directional pleiotropy. In our data it was 0.19 for menopause, meaning no evidence of it.

Weighted median

The weighted median is consistent even if up to half of the instruments are invalid, which neither IVW nor Egger tolerate.

```
ratios = by / bx
weights = (bx**2) / (sy**2)
```

- For each SNP, by / bx is its own little causal estimate (the “Wald ratio”).
- weights approximate the inverse variance of each Wald ratio; a strong, precisely-measured SNP gets more say.

```
def wm(r, w):
    order = np.argsort(r)
    r, w = r[order], w[order]
    cw = np.cumsum(w) - 0.5 * w
    cw /= np.sum(w)
    k = np.searchsorted(cw, 0.5)
    ...
    return r[k-1] + (r[k]-r[k-1]) * (0.5-cw[k-1]) / (cw[k]-cw[k-1])
```

- Sort the Wald ratios; walk up the cumulative weight; the weighted median is the ratio at which the cumulative weight passes 50%. The last line linearly interpolates between the two straddling SNPs so the answer is smooth.

```

for _ in range(n_boot):
    bxi = rng.normal(bx, sx)
    byi = rng.normal(by, sy)
    boots.append(wm(byi / bxi, (bxi**2) / (sy**2)))
se = float(np.std(boots))

```

- There is no neat formula for the weighted median's standard error, so we bootstrap: draw new bx/by from their sampling distributions many times, recompute the weighted median each time, and take the spread of those as the SE.

Cochran's Q

```

def cochrans_q(bx, sx, by, sy) -> dict:
    b = ivw(bx, sx, by, sy).estimate
    w = 1.0 / sy**2
    q = float(np.sum(w * (by - b * bx) ** 2))

```

- Q is the weighted sum of squared distances of the SNPs from the IVW line. If all SNPs agreed on one causal effect, Q would be about its degrees of freedom. A large Q (small p) signals heterogeneity, that is, some instruments are invalid.

2. Instruments: clumping and harmonization (mr/src/estrogen_mr/instruments.py)

Distance clumping

Genome-wide association hits cluster: many neighbouring SNPs tag the same signal because they are inherited together (linkage disequilibrium). Using all of them double-counts. Clumping keeps one representative per signal.

```

for _, g in df.sort_values(pval).groupby(chrom):
    chosen: list[int] = []
    for idx, r in g.iterrows():
        if all(abs(r[pos] - p) > window for p in chosen):
            chosen.append(r[pos])
    keep.append(idx)

```

- Sort by p-value (strongest first) and go chromosome by chromosome.
- Keep a SNP only if it is more than window base pairs from every SNP already kept. So the most significant SNP in each window wins and its neighbours are dropped. With window = 1e6 this keeps roughly independent signals. The gold-standard alternative uses a genotype reference to measure correlation directly; run_mr_ld_clump.py does that with PLINK, and it gave the same answer.

Allele harmonization

A SNP has two alleles. The exposure GWAS and the outcome GWAS may report their effect relative to different alleles, or even opposite DNA strands. If you do not line them up, a real effect

can flip sign and cancel out.

```
ea, oa = str(r["Effect_Allele"]).upper(), str(r["Other_Allele"]).upper()
oe, on_ = str(r["oa_eff"]).upper(), str(r["oa_non"]).upper()
b = r["beta_out"]
if {ea, oa} != {oe, on_}:
    continue # position collision / incompatible alleles
```

- ea/oa are the exposure's effect and other alleles; oe/on_ the outcome's.
- {ea, oa} != {oe, on_} compares them as sets. If the two studies do not even name the same pair of alleles at this position, they are not the same variant, so we drop it.

```
if (oe, on_) == (oa, ea):
    b = -b # allele swap
```

- Same alleles but swapped: the outcome reports its effect for the allele the exposure calls "other". Flipping the sign of beta_out re-expresses it for the exposure's effect allele, so both now speak about the same allele.

```
if {ea, oa} in _PALINDROMIC:
    eaf = float(r.get("EAF", np.nan))
    if not np.isnan(eaf) and abs(eaf - 0.5) < (0.5 - eaf_ambiguous):
        continue # ambiguous palindrome
```

- A/T and C/G SNPs are "palindromic": the allele pair reads the same on both DNA strands, so you cannot tell from the alleles alone which strand a study used. When the allele frequency is near 0.5 you also cannot use frequency to break the tie, so we drop those SNPs rather than risk a silent sign flip.

The function returns bx/sx/by/sy ready for the estimators above. f_statistic returns the mean of (bx/sx)^2, the instrument strength; below about 10 you worry about weak-instrument bias. Ours was near 98.

3. Multivariable MR (mr/src/estrogen_mr/mvmr.py)

When two exposures share instruments (SHBG and testosterone do), single-exposure MR cannot tell whose effect a SNP is really tagging. MVMR fits both at once.

```
Bx = np.asarray(Bx, dtype=float) # (n_snp x k) exposure effects
w = 1.0 / np.asarray(se_out) ** 2
W = np.diag(w)
xtwx = Bx.T @ W @ Bx
beta = np.linalg.solve(xtwx, Bx.T @ W @ by)
```

- Exactly IVW again, but Bx now has one column per exposure, so beta is a vector of direct effects: each exposure's effect holding the others fixed. This is why the SHBG estimate moved from 0.73 (alone) to 1.06 (with testosterone held fixed): the single-exposure signal was partly its shared testosterone.

```

resid = by - Bx @ beta
phi = max(1.0, float((resid * w) @ resid) / dof) # random-effects scaling
cov = np.linalg.inv(xtwx) * phi
se = np.sqrt(np.diag(cov))

```

- `phi` inflates the standard errors when the SNPs disagree with the fitted plane (multiplicative random effects), never shrinks them (the `max(1.0, ...)`). The diagonal of `cov` gives each exposure's SE.
-

4. Quality control, power, equivalence (`mr/src/estrogen_mr/qc.py`)

A guided tour; the code reads straightforwardly given section 1.

- `ivw_random`, IVW with the same multiplicative random-effects inflation as MVMR, used as the reported primary.
 - `steiger`, for each SNP, compares how much variance it explains in the exposure versus the outcome ($r^2 = F / (F + n - 2)$), and keeps SNPs that explain more of the exposure. This checks the arrow points exposure to outcome, not the reverse.
 - `mr_presso_global`, simulates outcome effects under the fitted model, recomputes the residual sum of squares many times, and asks whether the observed one is extreme. A small `p` means outlier pleiotropy.
 - `power_mde`, inverts the usual power formula. `mde = (z_alpha + z_power) * se` gives the smallest effect detectable at 80% power; the function reports it per year and per standard deviation. This is why we report a bound, not “no effect”.
 - `tost`, two one-sided tests. It asks separately whether the estimate is significantly inside a pre-set “smallest effect of interest” on the protective side and on the harmful side. We could reject strong protection (p 0.027) but not harm (p 0.53), so the honest statement is a bounded null, not equivalence.
-

5. The ICU identification argument (`icu/src/estrogen_asah_dci/analysis/multiverse.py`)

The observational arm's whole point is to show it *cannot* answer the question. Two functions carry that argument.

- `spec_curve`, loops over every defensible modelling choice (menopause age cutoff, covariate set, outcome definition, data source) and records the odds ratio for each. On real data 41 of 71 fits were significant and every one pointed away from protection. That pattern is the fingerprint of age acting on vasospasm, not of estrogen, because age both defines the exposure and independently raises vasospasm risk.
 - `age_sex_did`, fits a flexible age curve for the DCI risk and lets it differ by sex, then reads off the female-minus-male change across the menopausal transition. Men age without an abrupt hormone change, so they absorb the shared ageing gradient; a female-specific jump after menopause would be the one thing age cannot explain. There was none (odds-ratio ratio 1.04). That is the cleanest statement the ICU data can make, and it is null.
-

This walkthrough covers the scientific core. The extraction code (`icu/.../extract`), the codelist matchers (`icu/.../codelists.py`), and the pipeline scripts are documented in place and reproduce from the synthetic fixture without any data.